Journal of Systems Architecture xxx (xxxx) xxx

Contents lists available at ScienceDirect

Journal of Systems Architecture

journal homepage: www.elsevier.com/locate/sysarc

A Blockchain-based access control scheme with multiple attribute authorities for secure cloud data sharing

Xuanmei Qin *, Yongfeng Huang *, Zhen Yang, Xing Li

Department of Electronic Engineering, Tsinghua University, Beijing 100084, China
Beijing National Research Center for Information Science and Technology (BNRist), Beijing 100084, China

ARTICLE INFO

Keywords:
Attribute-based encryption
Blockchain
Access control
Multiple authorities
Data sharing

ABSTRACT

Ciphertext-policy attribute-based encryption(CP-ABE) has been widely studied and used in access control schemes for secure data sharing. Since in most of the existing attribute-based encryption methods, all user attributes are managed by a single central authority, it is easy to cause a single point of failure. Therefore, several multi-authority CP-ABE schemes are proposed to manage user attributes by multiple authorities. However, these schemes still do not eliminate the single point of failure in essence or suffer from high computation and communication overhead on data users. In this paper, we propose a Blockchain-based Multi-authority Access Control scheme called BMAC for sharing data securely. Shamir secret sharing scheme and permissioned blockchain (Hyperledger Fabric) are introduced to implement that each attribute is jointly managed by multiple authorities to avoid single point of failure. In addition, we take advantage of blockchain technology to establish trust among multiple authorities and exploit smart contracts to compute tokens for attributes managed across multiple management domains, which reduces communication and computation overhead on the data user side. Moreover, blockchain helps to record the access control process in a secure and auditable way. Finally, we analyze the security of the proposed algorithm. Further analysis and comparison show the performance of the proposed method.

1. Introduction

With the development of Internet technology and technological innovation, the cloud has significantly changed the way data is shared. It provides individuals and industries with various data storage and management services that enable customers to easily access resources and share data. A Global Industry Vision (GIV) whitepaper predicts that, by 2025,"every company everywhere will be using cloud technology and 85% of business applications will be cloud-based"[1].

While cloud providers put a lot of effort into ensuring the convenience of data sharing, there are still many security issues need to be considered. One of the main concerns is that cloud providers are not fully trusted. They will not only suffer external attacks from adversaries, but also internal attacks from malicious staff, which leads to huge data security risks for data sharing in cloud. For example, Azure security vulnerability result in the disclosure of 250M Customer Details [2], Amazon employees leak customers' personal data to third parties for personal interests [3], and it is not uncommon for cloud storage providers to snoop on users' private data. To address the above issues caused by not fully trusted cloud storage service providers, cryptographic access control schemes such as IBE [4], CP-ABE [5] and KP-ABE [6] are proposed to secure sharing data by encrypting it with a secret key, such that only users with the secret key can decrypt data.

Ciphertext-policy attribute-based encryption (CP-ABE) is considered as one of the most suitable technology to provide secure data access control in public cloud storage [7]. However, in most of existing CP-ABE schemes [7–9], all participants are forced to trust a single authority. It is easy to cause a single point of failure, and in practical applications, attributes are generally distributed across different trust domains and organizations. Therefore, Chase [10] propose the first multi-authority CP-ABE scheme, in which several disjoint domains are managed by different authorities to realize that user attributes are issued across multiple authorities. Although some extensions are also proposed [11–13], they still have the problem of the single-point of failure. Since in these schemes, the whole attribute set is divided into multiple disjoint subsets each of which is maintained by only a single authority. Li et al. [14] propose a TMACS scheme, taking advantage of threshold secret sharing to deal with the single-point bottleneck problem. Such that any one authority cannot obtain the master key by itself alone. Nevertheless, in this scheme, data users need to select multiple authorities from the authority list and communicate with each

* Corresponding authors.
E-mail addresses: qxm17@mails.tsinghua.edu.cn (X. Qin), yfhuang@tsinghua.edu.cn (Y. Huang).

https://doi.org/10.1016/j.sysarc.2020.101854
Received 29 March 2020; Received in revised form 17 July 2020; Accepted 3 August 2020
Available online 20 August 2020

of these authorities to request secret key shares, which brings additional computational and communicational overhead.

As a promising technology, blockchain has received much attention in recent years. Inspired by security properties of blockchain [15] and its innovative application in data communication [16,17] and sharing [18–23]. We intend to establish multi-party trust through blockchain, write the collaborative computing procedure among multiple authorities from multiple attribute domains into smart contracts, and generate decryption tokens for users. Since blockchain transactions have the property of transparency, it is necessary to consider how to integrate off-chain and on-chain algorithms to ensure the security of access control scheme based on blockchain.

In this paper, we propose a blockchain-based multi-authority access control scheme, named BMAC, for secure cloud data sharing to address the issues mentioned above such as multi-authority cross-domain collaboration, single point of failure and high computational and communicational overhead. In addition, a trustable and immutable access logs is recorded on blockchain, such that data owner can easily monitor users' access behavior. Overall, our main contributions are summarized as follows.

(1) A decentralized access control scheme based on blockchain and multi-authority attribute-based encryption, named BMAC, is proposed, which can solve the problems of a single point of failure, high computation and communication overhead on the data user side. In this scheme, the data access logs can be recorded on the blockchain, so as to realize auditable access control management.
(2) We extend the classical multi-authority attribute-based encryption method and design four smart contracts to realize multi-authority cross-domain collaboration. We take advantage of smart contracts to establish mutual trust between multiple authorities and collect attribute sub-tokens for collaborative calculations to generate a decryption token for users.
(3) We analyzed the security and performance of the scheme, and analyzed the effectiveness of our multi-authority ABE scheme from the perspective of communication overhead and computation overhead on the user side.

The rest of this paper is organized as follows. Section 2 reviews related works about multi-authority attribute-based access control schemes, along with the researches on blockchain-based access control. In Section 3, we introduce some technical preliminaries. The system model and security model are presented in Section 4. The details of our blockchain-based multi-authority access control scheme are present in Section 5. In Section 6, we analyze security properties and evaluate the performance. Finally, we made a conclusion in Section 8.

2. Related work

In this section, we would introduce some related works in terms of multi-authority attribute based encryption schemes and Blockchain-based access control schemes.

2.1. Multi-authority attribute based encryption schemes for access control

The problem of how to construct a scheme allowing multiple authorities to distribute attribute was first proposed by Sahai and Waters [24]. Chase proposed a multi-authority attribute based encryption scheme that utilizing a global identifier for tying users' keys distributing from multiple authorities together [10]. Lewko and Waters assumed that there is no coordination between authorities and proposed a new scheme to achieve that each attribute is managed by different authorities [11]. After that, several literatures have been proposed on multi-authority attribute based schemes. In order to reduce the computation and communication overhead, Yang et al. designed efficient decryption and immediate attribute revocation methods for multi-authority CP-ABE scheme [12]. Sandor et al. proposed an efficient multi-authority attribute based encryption scheme by introducing the cloud user assistant to improve the performance of encryption and decryption operations [13]. In these works, disjoint attribute subsets are independently managed by different authorities. The single-point of failure problem is still not solved. In order to solve this problem, Li et al. [14] utilized (t, n)-threshold secret sharing to share the master key among multiple authorities. This scheme, called TMACS, could realize that multiple authorities jointly manage a uniform attribute set. As in many scenarios, a resource provider wants to share data according to a policy over attributes issued across different domains, they also construct a hybrid scheme which combine Lewko and Waters's scheme with TMACS. However, in this scheme, data users need to select multiple authorities from the authority list and communicate with each of these authorities to request secret key shares, which brings additional computation and communication overhead on user side. Our method takes advantage of blockchain technology to facilitate multi-authority cross-domain management of attributes. Attribute token shares, or called attribute sub-tokens, are gathered through a smart contract to calculate the attribute token, and then compute a decryption token for data users. In this way, data users obtain decryption token from a smart contract, instead of communicating with different attribute authorities to gather secret key shares by themselves, so as to reduce their communication cost.

2.2. Blockchain-based access control schemes

The idea of taking advantage of blockchain transactions and smart contracts for access control has been preliminary studied in some literature. The security properties of blockchain make the access records on blockchain tamper-proof and auditable.

Zhu et al. [25] present a transaction-based access control model. In this model, four types of transactions, corresponding to four phases of access control procedure, are designed to carry out subject registration, object escrow, access request and grant. Maesa et al. [26] define blockchain transactions to create, update and revoke policies. And the access right obtained from a resource owner can be transferred from one user to another through a transaction. Then, any user can inspect these transactions to check the access control history. Paillisse et al. [27] utilize blockchain to conduct distributed access control in the situation with multi-administrative domains. Administrators specify group-based policies and store them into blockchain by transactions. Routers check if a user is authorized to access a resource through blockchain to establish a secure connection to resource user.

Guo et al. [28] propose a multi-authority attribute-based access control scheme based on smart contracts. A data user collects attribute certificates issued by multiple attribute authorities through different smart contracts. Once these attributes satisfy the access policy, the smart contract between the data owner and user is triggered, and the data user obtains a symmetric key encrypted by his/her public key for decryption of the shared data. Maesa et al. [26] design an access control system to manage XACML policies and execute the access decision process through smart contracts. Each policy is converted into smart contract and then deployed on blockchain. A data owner is able to audit the policy evaluation process for each access request. Gao et al. [29] utilize blockchain for traceability and accountability. A data user generates a proof and triggers a smart contract to verify its authority. Then data owner generates a secret key for the user to decrypt the ciphertext and send an access transaction to record the access control procedure on blockchain.

However, the above methods need to establish a specific contract or transaction between each pair of data owner and user. It brings the challenge of managing contracts or transactions when the resources increasing. In our proposed method, instead of establishing a specific contract or transaction for each pair of data owner and user, we can implement access control between different data owners and user by deploying only four smart contracts.

3. Preliminaries

In this section, we review some background information on bilinear maps and the security definition, then we briefly describe Shamir secret sharing, multi-authority CP-ABE scheme and blockchain introduced into BMAC.

3.1. Bilinear maps

Let $\mathbb{G}$, $\mathbb{G}_T$ be two multiplicative cyclic groups of prime order p, and g be a generator of $\mathbb{G}$. A bilinear map $e : \mathbb{G}\times\mathbb{G} \to \mathbb{G}_T$ has the following three properties:

(1) *Bilinearity:* $\forall a, b \in \mathbb{Z}_p$ and $\forall u, v \in \mathbb{G}$, we have $e(u^a, v^b) = e(u, v)^{ab}$.
(2) *Non-degeneracy:* $\forall u, v \in \mathbb{G}$ such that $e(u, v) \neq 1$, which means the map does not send any pair in $\mathbb{G}\times\mathbb{G}$ to the identity in $\mathbb{G}_T$.
(3) *Computability:* $\forall u, v \in \mathbb{G}$, there is an efficient algorithm to compute $e(u, v)$.

3.2. Shamir secret sharing (SSS) scheme

A shamir secret sharing scheme consists of the following algorithms:

(1) *Share:* To share a secret $s \in Z_p$ with threshold t among n participants, first build a $(t-1)$-degree polynomial $f(x) = a_0 + a_1 x + \cdots + a_{t-1}x^{t-1}$, where $a_0 = s, a_1, \ldots, a_{t-1}$ are randomly chosen from Z_p. Then, the shares for n participants are generated as $s_i = f(x_i)(i = 1, \ldots, n)$, where $x_i \in Z_p$.
(2) *Reconstruct:* To recover the secret s, a LaGrange interpolation is used to calculate s from any t of the n shares $s_1, s_2, \ldots, s_n$ following the equation $s = \sum_{i=1}^{t-1} s_i \prod_{m=0, m\neq j}^{t-1} \frac{x_m}{x_m - x_j}$

3.3. Multi-authority CP-ABE scheme

A classical multi-authority ciphertext-policy attribute-based encryption scheme is mainly composed of four algorithms, Setup, Encrypt, KeyGen and Decrypt. Here we adopt the definition and construction from Lewko A. and Waters B.[11].

$GlobalSetup(\lambda) \to GP$. The global setup algorithm takes in the security parameter λ and outputs global parameters GP for the system.

$AuthoritySetup(GP) \to (SK, PK)$. Each authority runs the authority setup algorithm with GP as input to produce its own secret key and public key pair, (SK, PK).

$Encrypt(M, (A, \rho), GP, PK) \to CT$. The encryption algorithm takes in a message M, an access matrix (A, ρ), public keys of relevant authorities, and the global parameters. It outputs a ciphertext CT.

$KeyGen(GID, GP, \omega, SK) \to (K_\omega, GID)$. The key generation algorithm takes in an identity GID, the global parameters, an attribute ω belonging to some authority, and the secret key SK for this authority. It produces a key (K_ω, GID) for this attribute, identity pair.

$Decrypt(CT, GP, \{K_\omega, GID\}) \to M$. The decryption algorithm takes in the global parameters, the ciphertext, and a collection of keys corresponding to attribute, identity pairs all with the same fixed identity GID. It outputs either the message M when the collection of attributes $\{\omega\}$ satisfies the access matrix corresponding to the ciphertext. Otherwise, decryption fails.

3.4. Blockchain

The blockchain is an immutable ledger maintained within a distribute network. In a blockchain-based access control system, the access logs are recorded as a series of transactions blocks, where each block is chained to the previous block by the hash value. In this paper, we do not need to care about the underlying technical specification of blockchain. The implementation of our scheme is based on Hyperledger Fabric. In order to describe clearly, we briefly review a few conceptions of Fabric blockchain.

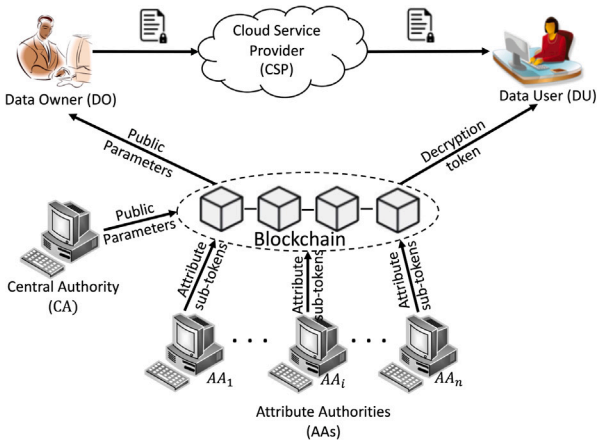

Fig. 1. Blockchain-based multi-authority access control system model.

Hyperledger Fabric: Hyperledger Fabric is an open source permissioned blockchain platform that provides a way to secure the interactions among a group of entities to build an agreement or framework for handling disputes. It supports pluggable consensus protocols that enable the platform to be more flexible to fit particular use cases and trust models. Which consensus protocol to choose will not be discussed in this paper. For a detailed analysis of different consensus protocols, some related studies [30] can be referred to. Fabric blockchain also provides privacy and confidentiality of transactions and the smart contracts through channel architecture and private data feature.

Transaction: The concept of transaction is introduced in Bitcoin for transferring cryptocurrencies between addresses. An address representing a user is a double hash of a public key derived from a ECDSA key pair. Each transaction is signed by the sender using private key and submitted to the blockchain. A transaction is approved when a process known as consensus completed.

Smart contract: A smart contract is a piece of code stored on blockchain, which can be self-verifying and self-executing once predefine conditions are triggered. It adopts Turing-complete programming language, which can implement any sophisticated logic we have grown accustomed to in our computers.

Endorser peers: Peer nodes are basic elements of Fabric blockchain. Each peer node maintains copies of ledgers and smart contracts. The endorser is a special role of peers. As each smart contract specifies an endorsement policy that refers to a set of endorser peers and defines conditions for a valid transaction endorsement, the endorser peers responsible for endorsing a transaction with respect to a particular smart contract before it is committed.

4. System and security model

This section provides an overview of the system. We briefly describe five participating entities and four phases of access control in the system model. We also introduce the syntax of the procedure flow to help understand the subsequent theoretical algorithms. Security assumption and model are given at the end of this section.

4.1. System model

As shown in Fig. 1, the proposed model contains five kinds of entities and a blockchain network.

Certificate authority (CA): The CA is responsible for the initialization of system by setting up system parameters and deploying smart contracts. Note that, it is not involved in the generation of AAs'

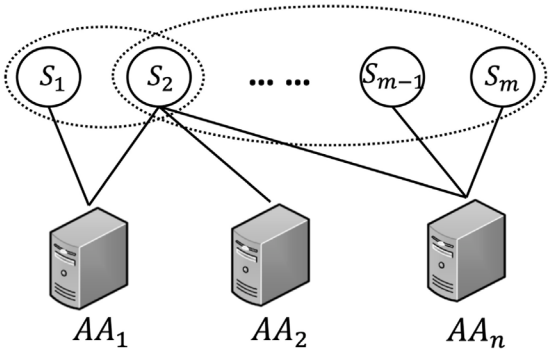

Fig. 2. Relationship between attributes and AAs.

master key sharing and users' secret key, and any operation related to attributes.

Attribute authorities (AAs): The AA is responsible for issuing attributes to data users through blockchain. There is a many-to-many mapping relationship between attributes and the attribute management authority AA. That is, each AA manages multiple attributes in one attribute domain, and each attribute can be managed by multiple AAs across domains. The attribute management model is shown in Fig. 2

Cloud service provider (CSP): The CSP provides data storage services for data owners. Here we assume that the CSP is untrusted.

Data owner (DO): The DO does not rely on the CSP but rather performs data access control based on cryptography. DOs set access policies, performs symmetric encryption on data file before uploading it to the CSP, and uploads hash value of data ciphertext and key ciphertext to blockchain.

Data user (DU): All DUs can download ciphertext freely from the CSP. When a user wants to access data, he/she needs to request a decryption token from the blockchain, and the decryption token is associated with a series of attributes. Users with different attributes have different decryption permissions. Only if the user's attributes satisfy the access policy embedded in the ciphertext can the decryption be successful.

Blockchain: The above entities all participate in a blockchain network. The blockchain stores public parameters and access metadata to ensure its integrity and immutable. It also assists entities to perform partial trusted computing, and enable multiple AA to collaboratively manage user attributes.

As each entity has a unique address in the blockchain network, which can be used as the global identity of entity. The global identity of AA is denoted as aid, and the global identity of DU is denoted as uid.

The system consists of four phases: **system initialization**, **encryption**, **token generation** and **decryption**. The procedure flow is shown in Fig. 3. The syntax for each procedure is as follows. The proposed algorithms actually can be divided into two types, off-chain and on-chain algorithm. The off-chain algorithm is executed locally by the client, and the on-chain algorithm is executed by smart contracts.

System initialization. The CA generates initialization parameter and uploads it to blockchain. Then the blockchain collects attribute public subkeys uploaded by different AAs to generate attribute public keys.

① CA first generates initialization system parameters and uploads them to blockchain.

$GlobalSetup(\lambda) \rightarrow GP$: The $GlobalSetup$ algorithm is run by CA. It takes security parameter λ as input. It outputs global parameters GP. Then the CA invokes smart contract to upload the public parameters GP to the blockchain.

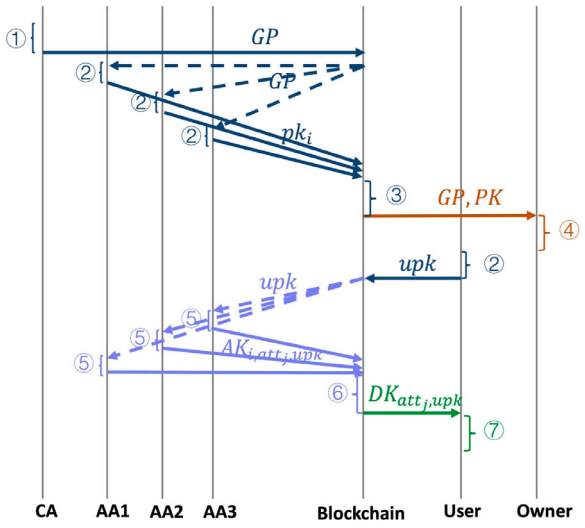

Fig. 3. Procedure flow of the system model. Different colors correspond to different phases.

② When AA and user join the system, they generate public and private key pairs respectively and upload public keys pk_i or upk to blockchain.

$AASetup(GP) \rightarrow (sk_i, pk_i)$: The $AASetup$ algorithm takes global parameters GP as input. It outputs a pair of key (sk_i, pk_i) for AA_i. The public key pk_i is upload to blockchain through a smart contract.
$UserSetup(GP) \rightarrow (usk, upk)$: The $UserSetup$ algorithm takes global parameters GP as input. It outputs a pair of keys (usk, upk) for a user. The public key upk is also upload to blockchain through smart contract.

③ Generate attribute public key through smart contract.

$onchain.SetupPK(pk_i) \rightarrow PK$: The $SetupPK$ algorithm is conduct by a smart contract. It takes multiple public keys pk_i which is from different AAs as input. It outputs public key of attributes PK.

Encryption. Data owner gets public parameters from the blockchain for encryption.

④ The DO performs symmetric encryption on data M, and obtains global parameters GP and attribute public keys PK from blockchain to encrypt the symmetric key KEY.

$Encrypt(KEY, (A, \rho), GP, PK) \rightarrow CT$: The DO performs $Encrypt$ algorithm. It takes message M, access structure (A, ρ), global parameters GP and a set of attribute public keys PK as input. It outputs ciphertext CT.

Token generation. When different AAs listen for the event that a user uid joins the system and uploads public key upk to the blockchain, they will generate and issue attribute sub-tokens to blockchain. The blockchain will gather and calculate a decryption token for the user.

⑤ AAs listen to user setup events and issue user attribute subtokens.

$SubAttrTokenGen(upk, sk_i, GP, att_j) \rightarrow AK_{i,att_j,uid}$: The sub-token generation algorithm takes user's public key upk, AA_i's secret key sk_i, global parameters GP and attribute att_j managed by the AA as input. It outputs an attribute sub-token $AK_{i,att_j,uid}$ for the user.

⑥ The blockchain collects and calculates user attribute keys $AK_{att_j,uid}$.

$onchain.AKGen(AK_{i,att_j,uid}, aid) \rightarrow AK_{att_j,uid}$: The attribute token generation algorithm is conducted by smart contract. It takes a set of

attribute sub-tokens $AK_{i,att_j,uid}$ and AAs' global identification aid as input. It outputs a set of attribute tokens $AK_{att_j,uid}$ for user with global identification uid.

$onchain.DKGen(AK_{att_j,uid}, CT_{KEY}.C_0) \rightarrow DK_{uid}$: The decryption token generation algorithm is conducted by smart contract. It takes user's attribute token and a component of key ciphertext as input. It outputs a decryption token DK_{uid} for user with global identification uid.

Decryption. The DU obtains the key ciphertext and attribute token from the blockchain for final decryption.

⑦ The data user uses the private key usk to perform final decryption.

$Decrypt(CT_{KEY}, DK_{uid}, usk, GP) \rightarrow KEY$: The decryption algorithm takes key ciphertext CT_{KEY}, decryption token DK_{uid}, user's private key usk and global parameters GP as input. It outputs symmetric key KEY.

4.2. Security model

In our proposed scheme, the security assumption of the five entities can be defined as follows. The **CSP** is assumed to be $not\ fully\ trusted$. It means that there may exist malicious staffs execute the tasks incorrectly assigned to them or collude with some malicious users to gain or tamper with the content of encrypted data for profits. The **CA** is a global trusted entity, but it can be comprised by an adversary. Similarly, each **AA** in our scheme is considered as trusted, but it also can be comprised by an adversary. Although **DU**s can freely get ciphertext of attribute key from the blockchain, they cannot decrypt the ciphertext unless their attributes satisfy the access policy embedded in the ciphertext. The **DO** in this work is considered trusted as it possesses the plain information. We assume communication channels are well protected by existing methods such as transport layer security (TLS) and we only focus on the confidentiality of the shared data.

We introduce the security model for multi-authority CP-ABE schemes, which is similar to Lewko and Waters [11]. It is assumed that the adversary in the security game can adaptively query for attribute sub-tokens that cannot be used to decrypt the challenge ciphertext, and the adversary can statically corrupt the attribute authority. The security game is described as follows.

Setup. The challenger perform the system initialization operation to generate the system parameter GP. The adversary specifies a set of corrupted authorities and can directly obtain public–private key pairs from the corrupted authorities

Key Query Phase 1. The adversary submits (S, uid) to the challenger for sub-token queries of an attribute set S, where uid is an identity. The challenger generates the corresponding sub-tokens $\{AK_{i,\omega,uid}\}_{\omega \in S}$.

Challenge. The adversary submits two messages of equal length, m_0 and m_1 and an access structure (M, ρ). The access structure needs to satisfy the constraint that it cannot be satisfied by the attributes obtained from corrupt authorities. The challenger randomly selects $b \in 0, 1$ and executes the encryption algorithm to encrypt message m_b under the access structure. And the ciphertext is returned to the adversary.

Key Query Phase 2. The adversary may repeat phase 1 to query more attribute tokens, as long as they do not satisfy the challenge access structure.

Guess. The adversary outputs a guess b' of b. If $b' = b$, then the adversary wins the game. The advantage of an adversary in the game is defined as $Adv_{\mathcal{A}} = Pr[b' = b] = 1/2$

Definition 1. The proposed multi-authority CP-ABE scheme is secure if all polynomial time adversaries have at most a negligible advantage in the above game.

attribute ω	*authority list* $\{aid_1, aid_2, \ldots, aid_{n_\omega}\}$	*threshold* t_ω

Fig. 4. Attribute management format.

5. A blockchain-based multi-authority access control scheme

The proposed scheme builds mutual trust among multiple AAs based on the blockchain, and utilizes smart contracts to implement cross-domain management of user attributes, such that users only need to interact with a smart contract to obtain attributes without communicating with multiple AAs. We begin by introducing smart contracts proposed in our scheme, and then give a detailed algorithm description for each stage of access control.

5.1. Smart contract deployment

In order to describe easily, four smart contracts are introduced: **attribute management contract**, **attribute public key generation contract**, **attribute token generation contract** and **decryption token generation contract**, which can also be written into one contract for ease of deployment. These contracts are deployed on the blockchain by the CA during the system initialization phase. Generally, a smart contract is invoked by a transaction. Let CN be the contract name and $Args$ be the input parameters of contract, a transaction can be denoted as $Tx = (CN, Args)$.

5.1.1. Attribute management contract (AMC)

In order to realize the cooperative management of attributes by multiple AAs, the CA first assigns multiple AAs for each attribute through the AMC contract during the system initialization phase. Each attribute has two main management parameters as the format in Fig. 4: a list of n_ω AAs indicates which AAs jointly manages an attribute ω and a threshold t_ω. A user must obtain t authorizations from n AAs to obtain the attribute.

Protocol 1 illustrates the process of authorities' assignment defined in AMC contract. The contract first verifies the identity of the sender, and only the CA can invoke AMC contract to assign the corresponding AAs for each attribute. Generally, in Fabric smart contracts, variables are stored in the form of key–value pairs, so we take the attribute ω as the key and its management parameters, authorities and threshold t as the values.

Protocol 1 Attribute management

```
procedure AssignAuthorities(ω, authorities, t)
    if CheckSender(sender, CA) =True then
        //Assign authority list to attribute ω,
        value[ω].authoritylist ← authorities;
        value[ω].threshold ← t;
    end if
end procedure
```

5.1.2. Attribute public key generation contract (PKC)

Since an attribute is jointly managed by multiple AAs, a trusted platform is required to gather attribute public subkeys generated by different AAs. During the system initialization phase, the PKC contract automatically calculates the attribute public key for encryption. The specific process is shown in protocol 2, after AAs upload the attribute public subkeys $PK_{i,\omega}$ in their respective management domains, the PKC contract executes $SetupPK$ algorithm to generate the attribute public key for encryption after collecting enough public subkeys PK_ω. The details of $SetupPK$ algorithm is described in Section 5.2.

Protocol 2 Attribute public key generation

procedure SetupPK(ω, $PK_{i,\omega}$)
  if CheckSender(sender, AA) =True **then**
    //collect public subkeys of attribute ω
    $subPK[\omega] \leftarrow subPK[\omega] \cup pk_i$;
    $count[\omega]++$;
    if $count[\omega] = value[\omega].threshold$ **then**
      //Generate attribute public key
      $PK_\omega \leftarrow SetupPK(subPK[\omega])$;
    end if
  end if
end procedure

5.1.3. Attribute token generation contract (ATC) and decryption token generation contract (DTC)

To generate tokens for decryption, two contracts, ATC and DTC, are involved. When the user completes the initial setting and joins the system, different AAs generate and distribute the attribute sub-tokens in their respective management domains to the ATC contract. After collecting t sub-tokens, the ATC contract automatically generates attribute tokens and issues them to users for final decryption. Protocol 3 illustrates the attribute token generation process that AA_i generate an attribute sub-token $AK_{i,\omega,uid}$ and upload it to ATC to authorize an attribute ω in its own attribute management domain to the user uid. After the contract has collected enough sub-tokens, the $AKGen$ algorithm is executed to generate the attribute token $AK_{\omega,uid}$. This process is executed in advance after user setup is completed. Protocol 4 illustrates the decryption token generation process. When the user initiates an access request, the DTC contract obtains the KEY ciphertext CT_{KEY} uploaded by the data owner and attribute token $AK_{\omega,uid}$, executes the $DKGen$ algorithm, and calculates the decryption token DK_{uid}. The details of the $AKGen$ algorithm and the $DKGen$ algorithm are described in Section 5.4.

Protocol 3 Attribute token generation

procedure GenAttributeToken(uid, $AK_{i,\omega,uid}$)
  if CheckSender(sender, AA) =True **then**
    //collect attribute sub-tokens
    $subAK[uid] \leftarrow subAK[uid] \cup AK_{i,\omega,uid}$;
    $count[\omega]++$;
    if $count[\omega] = value[\omega].threshold$ **then**
      //Generate attribute tokens
      $AK_{\omega,uid} \leftarrow AKGen(subAK[uid], \{aid\})$;
    end if
  end if
end procedure

Protocol 4 Decryption token generation

procedure GenDecToken(uid, $dataHash$)
  // generate decryption token after receiving request from user
  $CT_{KEY} \leftarrow getCT(dataHash)$
  $DK_{uid} \leftarrow DKGen(AK_{\omega,uid}, CT_{KEY}.C_0)$;
end procedure

5.2. System initialization

Global setup. Let G_1 be a multiplicative group of prime order p, g is a generator of G_1, then define a bilinear map $e: G_1 \times G_1 \rightarrow G_2$. Let $H: \{0,1\}^* \rightarrow G_1$ be one-way hash function. It maps attributes to elements of G_1. The global public parameters $GP = (p, g, H)$. The CA invokes the attribute management contract to set management parameters for each attribute, specifying that each attribute $\omega \in Att_1, \ldots, Att_U$ is managed by n_ω AA. As long as there are t_ω out of n_ω AAs issue attribute subkeys, the user can obtain attribute authorization.

AA setup. For each attribute managed jointly by a list of AAs, Shamir's secret sharing scheme is adopted for these AAs to cooperate with each other to obtain sub-keys. Suppose that an attribute ω is managed by n_ω AAs, where each AA_i selects two random number $\alpha_i, \beta_i \in Z_p$ as its sub-secret and two random polynomials $f_i(x)$ and $h_i(x)$ of $(t_\omega - 1)$-degree. Such that the master key is implicitly defined as

$$\alpha_\omega = \sum_{i=1}^{n_\omega} \alpha_i, \qquad \beta_\omega = \sum_{i=1}^{n_\omega} \beta_i. \tag{1}$$

Let $\alpha_i = f_i(0), \beta_i = h_i(0)$. Then AA_i calculates the shares $s_{ij} = f_i(aid_j)$, $s'_{ij} = h_i(aid_j)$ for each of the other AAs, and sends the shares (s_{ij}, s'_{ij}) to AA_j with global identity aid_j securely.

After receiving $n_\omega - 1$ shares (s_{ji}, s'_{ji}) from the other $n_\omega - 1$ AAs, AA_i calculates its master share keys $sk_{i,\omega} = (sk_{\alpha_\omega,i} = \sum_{j=1}^{n_\omega} s_{ji}, sk_{\beta_\omega,i} = \sum_{j=1}^{n_\omega} s'_{ji})$ and attribute public share keys $pk_{i,\omega} = (e(g,g)^{sk_{\alpha_\omega,i}}, g^{sk_{\beta_\omega,i}})$. The attribute public share keys can be shared with any other entities. For clarity, the master share keys and attribute public share keys are called attribute private subkeys and attribute public subkeys, respectively.

onchain.SetupPK. Since an attribute is jointly managed by multiple AAs, each attribute public key is determined by multiple attribute public subkeys generated by different AAs. After the setup stage, for an authority AA_i manages a set of attribute S_i, it will generate subkeys $(sk_{i,\omega}, pk_{i,\omega})_{\omega \in S_i}$. The public subkeys $pk_{i,\omega}$ are uploaded to blockchain through PKC contract. When a sufficient number of attribute public subkeys are collected, the PKC contract automatically executes $onchain.SetupPK$ algorithm, performing the following calculation for each attribute

$$e(g,g)^{\alpha_\omega} = \prod_{i=1}^{t_\omega} pk_{i,\omega,1}^{\prod_{k=1,i\neq k}^{t_\omega} \frac{aid_k}{aid_k - aid_i}}, \qquad g^{\beta_\omega} = \prod_{i=1}^{t_\omega} pk_{i,\omega,2}^{\prod_{k=1,i\neq k}^{\omega} \frac{aid_k}{aid_k - aid_i}} \tag{2}$$

where $pk_{i,\omega,1} = e(g,g)^{sk_{\alpha_\omega,i}}$, $pk_{i,\omega,2} = g^{sk_{\beta_\omega,i}}$. Then the public key of attribute ω is $PK_\omega = (e(g,g)^{\alpha_\omega}, g^{\beta_\omega})$, where $MK = (\alpha_\omega, \beta_\omega)$ is implicit attribute private key.

User setup. For each user joining the system, it firstly selects random number $usk \in \mathbb{Z}_p$ and compute a public key $upk = g^{usk}$ for user registration. To obtain attribute authorizations, users need to upload the public key to blockchain.

After the system initialization is completed, the attribute public key and user public key are stored in the blockchain.

5.3. Encryption

In this phase, a data owner first encrypts the data msg symmetrically. Let the symmetric key be KEY, then the data owner obtains public parameters GP and attribute public parameters PK from the blockchain and performs attribute-based encryption on the symmetric key. The ciphertext CT_{KEY} in this article refers to the ciphertext of the symmetric key. According to the literature [5], an access policy can be expressed as LSSS access structure (M, ρ). M denote an $m \times l$ matrix. The function ρ maps the xth row of M to an attribute $\rho(x) \in Att_1, \ldots, Att_U$. For each row of M, data owner chooses a random vector $v = (s, y_2, y_3, \ldots, y_l) \in \mathbb{Z}_p$, where s is a secret value and $y_2, y_3, \ldots, y_l$ are used to share secret s. Let $\lambda_x = M_x v^\intercal$, where M_x denotes the xth row of M. It also chooses a random vector $u = (0, u_2, u_3, \ldots, u_l) \in \mathbb{Z}_p$, computes $c_x = M_x u^\intercal$. And random numbers $r_1, r_2, \ldots, r_m \in \mathbb{Z}_p$ are chosen. Then ciphertext is calculated as

$$\begin{aligned} CT_{KEY} =& (C_0 = KEY \cdot e(g,g)^s, \\ & C_{1,x} = e(g,g)^{\lambda_x} \cdot e(g,g)^{\alpha_{\rho(x)} r_x}, \\ & C_{2,x} = g^{r_x}, C_{3,x} = g^{c_x} \cdot g^{\beta_{\rho(x)} r_x}, \forall x) \end{aligned} \tag{3}$$

Finally, the owner sends ciphertext CT_{KEY} to blockchain for data access.

5.4. Token generation

In order to reduce the user's communication burden, when a user uid completes the initialization setting, different AAs generate and issue the user's attribute sub-tokens in advance through the blockchain, and the ATC contract automatically calculates the user's attribute token. Then, when the user initiates an access request, the DTC contract obtains the ciphertext uploaded by the data owner and attribute token from blockchain, and generates a decryption token for user's final decryption. The details are as follows.

SubTokenGen: When the user setup is completed, AA_i execute the sub-token generation algorithm to calculate attribute sub-tokens for users in their own attribute management domain. The attribute sub-tokens are denoted as

$$AK_{i,\omega,uid} = upk^{sk_{\alpha_\omega,i}} H(uid)^{sk_{\beta_\omega,i}}, \tag{4}$$

where $sk_{\alpha_\omega,i}$ and $sk_{\beta_\omega,i}$ are attribute private subkeys. Then, AA_i call the ATC contract to store the sub-tokens to the blockchain.

onchain.AKGen: After gathering enough number of attribute sub-tokens, that is t_ω attribute sub-tokens, issued to the user uid, the ATC contract automatically generates an attribute token in advance. The attribute token is computed as

$$AK_{\omega,uid} = \prod_{i=1}^{t_\omega} AK_{i,\omega,uid}^{\prod_{k=1,i\neq k}^{t_\omega} \frac{aid_k}{aid_k - aid_i}} = upk^{\alpha_\omega} H(uid)^{\beta_\omega} \tag{5}$$

onchain.DKGen: When the user initiates an access request, the DTC contract utilized the key ciphertext CT_{KEY} and attribute tokens $AK_{\omega,uid}$ stored in blockchain to compute a decryption token DK_{uid}. Assume that a user possesses the attribute tokens $AK_{\rho(x),uid}$ for an attribute set S_U satisfying the access structure (M,ρ), then there exists parameters $t_x \in \mathbb{Z}_p$ such that $\sum_x t_x M_x = (1,0,\ldots,0)$, where M_x corresponds to an attribute in S_U, t_x can help the user to find the secret parameter s. For each attribute in S_U, the decryption token is denoted as $DK_{uid} = (C_0, L, R)$, where

$$L = \prod_x C_{1,x}^{t_x}, R = \prod_x \left(\frac{e(H(uid), C_{3,x})}{e(AK_{\rho(x),uid}, C_{2,x})}\right)^{t_x}. \tag{6}$$

It is returned to user for final decryption.

5.5. Decryption

The decryption algorithm is performed by data users. A data user obtains CT_{data} from cloud, and gets decryption token DK_{uid} from blockchain. Only when the user's attributes satisfy the ciphertext access policy can the decryption token be obtained to complete the final decryption and obtain the symmetric key KEY.

Specifically, after obtaining decryption token $DK_{uid} = (C_0, L, R)$ from contract, the user computes

$$\begin{aligned} C_U &= L \cdot R^{1/usk} \\ &= \prod_x \left(C_{1,x} \cdot \left(\frac{e(H(uid), C_{3,x})}{e(AK_{\rho(x),uid}, C_{2,x})}\right)^{1/usk}\right)^{t_x} \\ &= \prod_x \left(e(g,g)^{\lambda_x} e(H(uid), g)^{c_x/usk}\right)^{t_x} \end{aligned} \tag{7}$$

Since $\lambda_x = M_x v^\intercal, v = (s, y_2, y_3, \ldots, y_l)$ and $\sum_x t_x M_x = (1,0,\ldots,0)$

$$\sum_x t_x \lambda_x = \sum_x t_x M_x \cdot v^\intercal = (1,0,\ldots,0) \cdot (s, y_2, y_3, \ldots, y_l)^\intercal = s \tag{8}$$

Since $c_x = M_x u^\intercal$ and $u = (0, u_2, u_3, \ldots, u_l)$

$$\sum_x t_x c_x = \sum_x t_x M_x \cdot u^\intercal = (1,0,\ldots,0) \cdot (0, u_2, u_3, \ldots, u_l)^\intercal = 0 \tag{9}$$

then $C_U = e(g,g)^s$, the symmetric key is computed as

$$C_0 / C_U = KEY \cdot e(g,g)^s / e(g,g)^s = KEY \tag{10}$$

with the symmetric key, the user can finally get the plaintext.

6. Security and performance analysis

6.1. Security analysis

In this section, we give the security analysis of our blockchain-based multi-authority ABE for data sharing. We will prove the confidentiality of our proposed scheme under the not fully trusted cloud assumption, and revisit details that make our access control scheme secure. We will additionally prove that our scheme exhibits auditable access control. We provide our proof of security in Appendix.

6.1.1. Confidentiality guarantee

The CSP is responsible for storing data ciphertext, although the cloud server is able to obtain ciphertext freely, it does not have any advantages compared with malicious users.

The attribute tokens for users are collected by smart contract. As we adopt Fabric blockchain, which is a permissioned platform supporting transaction privacy and private data, these tokens are only available for AAs. What is more, these tokens are associate with user's public key upk, even if an adversary compromises some AAs and gets these tokens, it can still not decrypt successfully without the private key usk.

6.1.2. Security against collusion attack

When some malicious users collude with each other, they may share their secret attribute keys $e(H(uid), g)^{c_x}$ calculated from decryption token DK_{uid} and secret key usk to gain additional privilege. As the decryption token is associate with a specific user, if two users with different identities uid and uid' attempt to collude and combine their secret attribute keys, $e(H(uid), g)^{c_x}$ and $e(H(uid'), g)^{c'_x}$, they will not recover the factor $e(g,g)^s$. Thus, it is impossible for malicious users to collude and gain more privilege.

6.1.3. Security against compromising AAs

It is possible that an adversary compromises multiple AAs to gain some privileges. We introduce Shamir secret sharing scheme to share the secret keys among multiple AAs, which can guarantee that less than t out of n AAs secret key shares cannot learn anything about the secret key. Such that the adversary has to compromise more than t AAs to obtain secret key shares for illegal benefits. As Li et al. [14] analyzed that the overlarge value of t will only bring extra overhead, to increase the system security, we can set an appropriate threshold t to make the system secure against the attack with a high probability.

6.1.4. Auditable access control

In our blockchain-based scheme, when a user initiates access request through blockchain transaction, a trustable and immutable access log will be recorded on the chain. Both data owner and user are able to verify how the access policy has been evaluated for each access request that has been performed. Therefore, our scheme inherits the auditability property from blockchain technology.

7. Performance analysis

We conduct simulations to evaluate the performance of the proposed scheme and make a comparison between TMACS scheme [14] and our proposed. The scheme is implemented with a 256-bit BN elliptic curve. All the experiments were done on Mac with macOS Catalina 10.15, 1.6 GHz Intel Corei5 and 8G RAM.

Table 1
Communication cost.

Method	System initialization			Key/token distribution	
	CA	Each AA	User	Each AA	User
TMACS [14]	$(N_u + N_A)\|G_1\|$	$(2N_A - 1)\|G_1\| + (\|G_1\| + \|G_2\|)N_{attr}$	$\|G_1\|$	$N_{attr}\|G_2\| + 2\|G_1\|$	$t(N_{attr}\|G_2\| + 2\|G_1\|)$
Our method	$3\|G_1\| + \|SC\|$	$(2N_A - 1)\|G_1\| + (\|G_1\| + \|G_2\|)N_{attr}$	$\|G_1\|$	$N_{attr}\|G_2\| + 2\|G_1\|$	$3\|G_2\|$

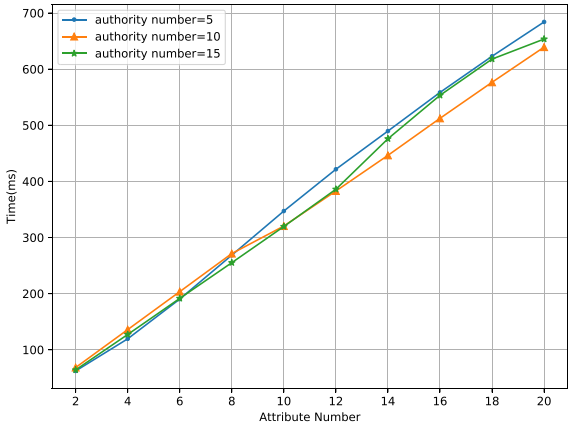

Fig. 5. The average execution time of $AASetup$ algorithm of single AA.

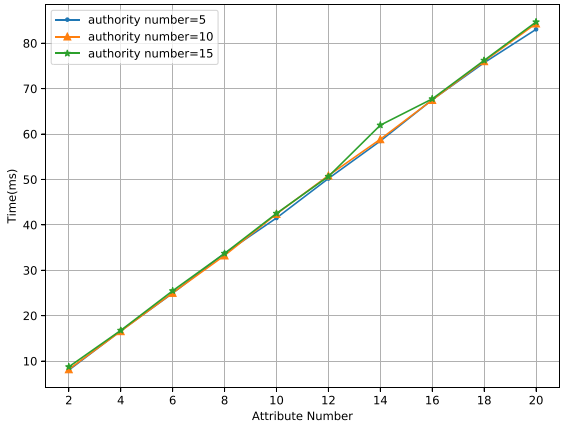

Fig. 6. The execution time of $SubTokenGen$ algorithm of single AA.

7.1. Communication overhead

For clarity of description, we have the following definitions: Let $|G_1|$ denote the size of element in G_1 and $|G_2|$ be the size of element in G_1. N_u denotes the number of users in the system. N_A denotes the number of AAs in the system. $|SC|$ denotes the size of the smart contracts. N_{attr} denotes the average number of attributes managed by AAs. N_{own} denotes the average number of the attribute set owned by users. t denotes the threshold of the number of AAs to recover an attribute.

Table 1 shows the comparison about communication overhead between our scheme and the TMACS scheme. During the System initialization phase, multi-authority settings inevitably introduce some overhead for AA registration, which is same as TMACS scheme. In our scheme, CA only need to deploy smart contract and upload the public parameter to blockchain, so the communication overhead is $3|G_1|+|SC|$. In the token distribution phase, the smart contract returns $DK_{uid} = (C_0, L, R)$ to the user. The size of element DK_{uid} is equal to $3|G_2|$. The communication overhead on user is a constant and obviously smaller than TMACS due to the fact that our proposed need not to communicate with different AAs to collect attribute keys.

7.2. Computation overhead

Figs. 5 and 6 show the computation overhead that multiple authorities jointly manage attributes. We considered the most complicated situation that each attribute is jointly managed by all authorities, the value of threshold is equal to the number of AAs and the access policy is a combination of all attributes with AND such as "attr1 AND attr2 AND attr3". Each experiment performs 10 times. As we can see, the average execution time of $AASetup$ algorithm and $SubTokenGen$ algorithm for each AA grows linearly as the number of attribute increases from 2 to 20. Furthermore, with increasing the number of AAs from 5 to 15, the running time of a single AA is not affected much.

Fig. 7 shows the computation overhead of decryption on the user side between our scheme and the TMACS scheme [14]. In this experiment, the authority number is set to be 3. With the increase of attribute number, compared with TMACS scheme in which the decryption time increases rapidly with the increase of attribute number, our scheme revolves around 11 ms regardless of the number of attributes for users

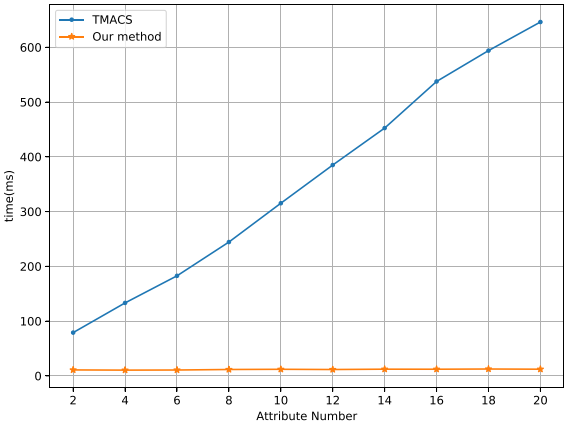

Fig. 7. The comparison between TMACS scheme and our method over the decryption time of user.

to decrypt ciphertext. The reason our method has better decryption performance on the user side is that the blockchain undertakes the main computing tasks. During the token generation phase, the DTC contract obtains the ciphertext uploaded by the data owner and attribute token from blockchain, and calculates a decryption token. After obtaining the decryption token, the user only needs to perform an exponential operation and simple multiplication operations to complete the final decryption, the execution time of which is independent of the number of attributes. Therefore, the decryption time on the user side is basically a constant.

7.2.1. Smart contract latency

A Fabric blockchain network is built based on Docker to validate the effectiveness of smart contracts. To measure the performance of the smart contracts, we utilize a benchmark tool called Caliper,[1] which is a blockchain performance testing tool that supports transaction latency, transaction throughput and other performance analysis [31].

[1] https://github.com/hyperledger/caliper/.

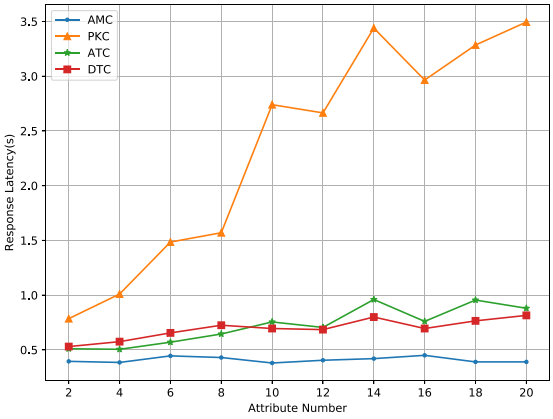

Fig. 8. The average response latencies of three smart contracts with the number of attribute increase.

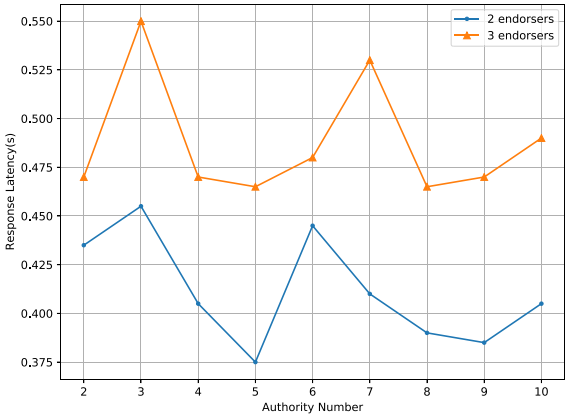

Fig. 9. The average response latencies of the AMC contract with the number of AAs and the number of endorsers increase.

Fig. 8 presents the response latency of four smart contracts. In this experiment, the number of authority is set to be 3 and there are 2 endorser peers in the blockchain networks. As we can see, the latencies grow linearly with the number of attributes. Although the delay of PKC contract is obviously longer than others due to the fact that each AA manages multiple attributes in one attribute domain, and each attribute is managed by multiple AAs across domains, it only self-executes once during system initialization phase. When a user initiates an access request, the smart contracts execute the DTC contract to generate and issue a decryption token. With the increase of attribute number, the response delay of DTC contract grows slowly, which is acceptable. Since the ATC contract will be automatically executed in advance after meets certain conditions, it will not affect users' access.

Fig. 9 shows the average response latency versus the number of AAs and endorsers when the number of attributes is 10. From the figure, we can see that the average response latency is basically steady. The minimum response time is 0.375 ms and the maximum response time is 0.455 s with the increase of the number of AAs when the number of endorsers is 2. Whereas, with 3 endorsers, the average response time increases and reaches about 0.48 s. Fig. 10 shows the average response time of four smart contracts with 3 endorsers is obviously longer than the average latency with 2 endorsers, the network scale has a negative impact on latency. So how to configure a suitable network scale to balance security and efficiency will be carried out in the our future work.

8. Conclusion

In this paper, we propose a multi-authority attribute-based access control scheme based on blockchain, named BMAC, for data sharing. Taking advantage of blockchain technology, our scheme achieves that each attribute is managed across different domains, eliminates the single-point bottleneck of the existing multi-authority CP-ABE schemes and reduces computation and communication overhead between the user and the multiple attribute authorities. The security analysis shows that our scheme could effectively resist to individual and colluded malicious users, as well as the not fully trusted cloud servers. Besides, with the blockchain-based scheme, trustable and immutable access logs are recorded on blockchain, such that data owner can easily monitor users' access behavior.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the National Key R&D Program of China [grant number 2016YFB0800402]; the National Natural Science Foundation of China [grant numbers U1836204, U1705261].

Appendix. Security proof

We prove our multi-authority access control is secure under the generic bilinear group and the random oracle models used in [5,11].

The generic bilinear group model. We follow [11] here. Let ψ_0 and ψ_1 be two injective maps from $\mathbb{Z}_p$ to $\{0,1\}^m$, where $m > 3log(p)$. We define the groups $G_0 = \psi_0(x) : x \in \mathbb{Z}_p$ and $G_1 = \psi_1(x) :\in \mathbb{Z}_p$. Assume that we are given oracles to compute the induced group operations on G_0, G_1 and an oracle to compute a non-degenerate bilinear map $e : G_0 \times G_0 \to G_1$. We refer to G_0 as a generic bilinear group.

To prove the security, we use the following theorem.

Theorem 1. *For any adversary $\mathcal{A}$, let q be a bound on the total number of group elements it receives from queries it makes to the oracles for the hash function, groups G_0 and G_1, and the bilinear map e, and from its interaction with the security game. Then the advantage of the adversary in the security game is $O(q^2/p)$.*

Proof. Suppose there is an adversary with non-negligible advantage $\epsilon = Adv_{\mathcal{A}}$ in the defined security game against our construction. It chooses a challenging matrix M with the dimension at most $q-1$ columns. With constraints in the security game, we can build a simulator $\mathcal{B}$ that plays the decisional q-parallel BDHE problem with non-negligible advantage. The detailed proof is described as follows.

We first consider the same modified game employed in [5,11]. In the security game, the challenge ciphertext has a component C_0. The adversary must distinguish between $C_0 = m_0 e(g,g)^s$ and $C_0 = m_1 e(g,g)^s$. We can instead consider a modified game in which the attacker must distinguish C_0 is $e(g,g)^s$ or $e(g,g)^t$, t is selected randomly from $\mathbb{Z}_p$.

Setup. The simulator $\mathcal{B}$ runs the GlobalSetup algorithm and outputs g as the public generator of G_1 and p as the group order. The adversary $\mathcal{A}$ specifies a set of corrupted authorities. For each attribute ω belonging to uncorrupted authorities, $\mathcal{B}$ runs the AASetup algorithm and gives $\mathcal{A}$ the public attribute keys $(e(g,g)^{\alpha_\omega}, g^{\beta_\omega})$.

Key Query Phase 1. When $\mathcal{A}$ makes a key query (S, uid), $\mathcal{B}$ first chooses a random exponent $h_{uid} \in \mathbb{Z}_p$ and sets $H(uid) = g^{h_{uid}}$. $\mathcal{B}$ stores this value so that it can respond consistently if $H(uid)$ is queried again. Then, $\mathcal{B}$ generates the corresponding tokens $AK_{\omega,uid}$ using the token generation algorithm.

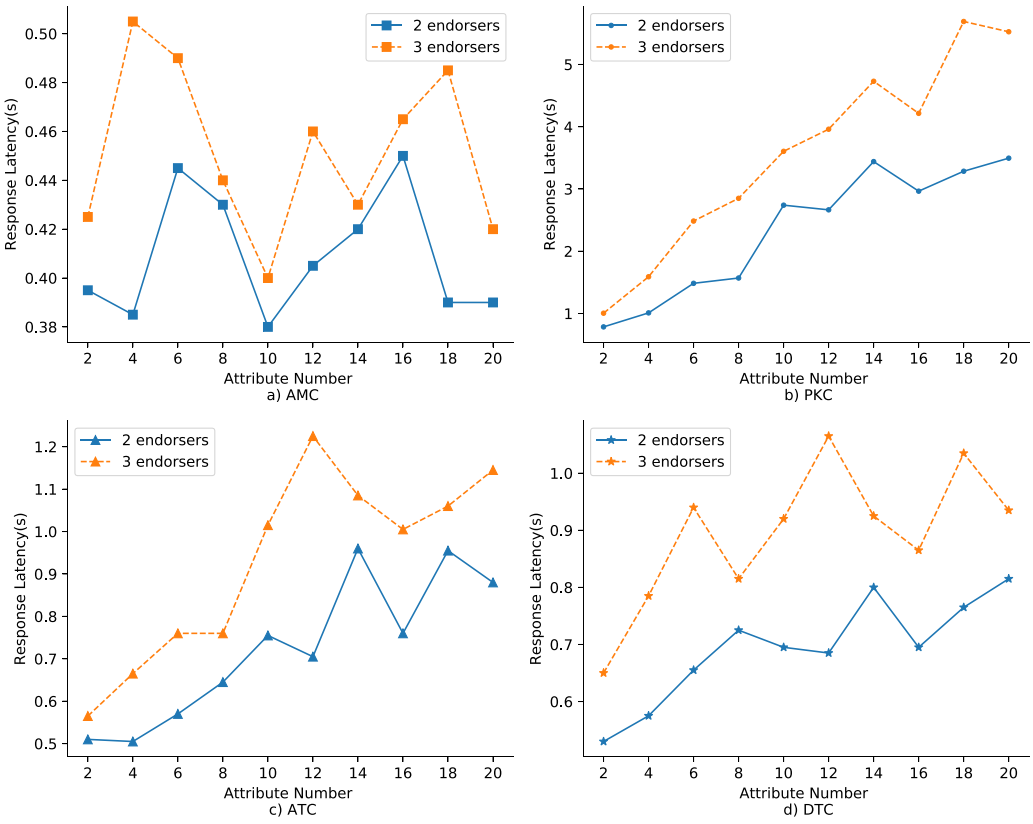

Fig. 10. The average response latencies of four smart contracts with the number of endorser node increase.

Challenge. To produce the challenge ciphertext, $\mathcal{B}$ chooses a random $s \in \mathbb{Z}_p$ and sets C_0. $\mathcal{B}$ also chooses two vectors, $v = (s, y_2, y_3, \ldots, y_l) \in \mathbb{Z}_p, u = (0, u_2, u_3, \ldots, u_l) \in \mathbb{Z}_p$, where $y_2, y_3, \ldots, y_l$, $u_2, u_3, \ldots, u_l$ are chosen randomly from $\mathbb{Z}_p$. Let $\lambda_x = M_x v^\intercal$, $c_x = M_x u^\intercal$. The simulator also chooses random values $r_x \in \mathbb{Z}_p$ for each row M_x of M, and a random value t from $\mathbb{Z}_p$. Using the group oracles, the simulator $\mathcal{B}$ uses the encryption algorithm to compute

$$C_0 = e(g,g)^t, C_{1,x} = e(g,g)^{\lambda_x} \cdot e(g,g)^{\alpha_{\rho(x)} r_x}, C_{2,x} = g^{r_x}, C_{3,x} = g^{c_x} \cdot g^{\beta_{\rho(x)} r_x}, \forall x$$

The challenge ciphertext is given to $\mathcal{A}$.

Key Query Phase 2. Same as Phase 1.

Guess. We will argue that will all but negligible probability, the adversary's view in the simulation is identically distributed to what its view would have been if C_0 had been set to $e(g,g)^s$ instead of $e(g,g)^t$. Therefore, there is no adversary that can have a non-negligible advantage in the modified security game. The adversary $\mathcal{A}$ hence cannot attain a non-negligible advantage in the real security game.

When the adversary makes a query to the group oracles, we may condition on the event that the adversary only provides as input values it received from the simulation, or intermediate values it already obtained from the oracles, and we think of each oracle query as a multivariate polynomial in the variable $t, \alpha_i, \beta_x, \lambda_x, r_x, c_x, g^{h_{uid}}$. Since ψ_0 and ψ_1 are random injective map from $\mathbb{Z}_p$ into a set of $> p^3$ elements, the probability of the attacker being able to guess an element in the image of ψ_0, ψ_1 which it has not previously obtained is negligible.

Now we consider what the adversary's view would have been if we substitute s for t. We will show that subject to the conditioning above, the adversary's view would have been identically distributed. Since the only way that the adversary's view can differ in the case of $t = s$ is if there are two queries f and f' into G_1 such that $f \neq f'$ but $f|_{t=s} = f'|_{t=s}$. Since t only appears as $e(g,g)^t$, the only queries the attacker can have on t are of the form θt, where θ is a constant. This implies that $f - f' = \theta s - \theta t$ for some constant θ. The adversary can then make a query of $f - f' + \theta t = \theta s$. We will now show that the adversary cannot make a query for $e(g,g)^{\theta s}$ and therefore arrive at a contradiction.

Table A.2
Possible query terms.

α_ω	β_ω	h_{uid}
$\alpha_\omega + h_{uid}\beta_\omega$	$\lambda_x + \alpha_{\rho(x)} r_x$	r_x
$c_x + \beta_{\rho(x)} r_x$	$\beta_\omega h_{uid}$	$\beta_{\omega'}\alpha_\omega + \beta_{\omega'} h_{uid}\beta_\omega$
$\beta_\omega r_x$	$\beta_\omega c_x + \beta_\omega \beta_{\rho(x)} r_x$	$h_{uid'}\alpha_\omega + h_{uid'} h_{uid}\beta_\omega$
$h_{uid} r_x$	$h_{uid} c_x + h_{uid}\beta_{\rho(x)} r_x$	$r_x\alpha_\omega + r_x h_{uid}\beta_\omega$
$\beta_\omega\beta_{\omega'}$	$h_{uid} h_{uid'}$	$r_x r_{x'}$
$(\alpha_\omega + h_{uid}\beta_\omega)(c_x + \beta_{\rho(x)} r_x)$	$r_x c_{x'} + r_x \beta_{\rho(x')} r_{x'}$	$(\alpha_\omega + h_{uid}\beta_\omega)(\alpha_{\omega'} + h_{uid'}\beta_{\omega'})$
$(c_x + \beta_{\rho(x)} r_x)(c_{x'} + \beta_{\rho(x')} r_{x'})$		

By analyzing the values given to the adversary during the simulation, we enumerate over all queries possible by means of the bilinear map and the group elements given to the adversary in Table A.2.

Recall that $\lambda_x = M_x v^\intercal$, where $v = (s, y_2, y_3, \ldots, y_l)$. $\lambda_x + \alpha_{\rho(x)} r_x$ is the only appearances of s in the above table. The only way that the adversary can create a term containing θs is by choosing constants θ_x to form $\sum_x \theta_x(\lambda_x + \alpha_{\rho(x)} r_x)$.

In order for the adversary to obtain a query polynomial of the form θs, the adversary must add other linear combinations in order to cancel the terms of the form $\theta_x \alpha_{\rho(x)} r_x$. The adversary can construct a query polynomial of the form $-\theta_x(r_x\alpha_{\rho(x)} + r_x h_{uid}\beta_{\rho(x)})$. The extra term $-\theta_x r_x h_{uid}\beta_{\rho(x)}$ can be canceled by using $\theta_x(h_{uid}c_x + h_{uid}\beta_{\rho(x)} r_x)$. Then, $\theta_x h_{uid} c_x$ is the remaining term.

However, there is no term that the adversary has access to that can cancel this term. Since this term will only cancel if the set of secret shares λ_x do allow for the reconstruction of the secret s and the attacker's obtained keys pass the challenge access structure. If this condition is satisfied, then the attacker has broken the rules of the security game. Therefore, any adversary query polynomial of this form cannot be of the form $\theta_x s$. Therefore, under conditions that hold with

all but negligible probability, the attacker's view when t is random is the same as the attacker's view when $t = s$. Therefore, the adversary cannot attain non-negligible advantage in the security game.

References

[1] HUAWEI, Touching an intelligent world, 2019, https://www.huawei.com/minisite/giv/Files/whitepaper_en_2019.pdf, [online, accessed 21/3/2020].

[2] R. Jennings, Microsoft leaks 250m customer details in azure fat-finger faux pas, 2020, https://securityboulevard.com/2020/01/microsoft-leaks-250m-customer-details-in-azure-fat-finger-faux-pas/, [online, accessed 21/3/2020].

[3] A. Palmer, Amazon fires employees for leaking customer email addresses and phone numbers, 2020, https://www.cnbc.com/2020/01/11/amazon-fires-employees-for-leaking-customer-email-addresses-and-phone-numbers.html/, [online, accessed 21/3/2020].

[4] D. Boneh, M. Franklin, Identity-based encryption from the weil pairing, in: J. Kilian (Ed.), Advances in Cryptology — CRYPTO 2001, Springer Berlin Heidelberg, Berlin, Heidelberg, 2001, pp. 213–229.

[5] J. Bethencourt, A. Sahai, B. Waters, Ciphertext-policy attribute-based encryption, in: 2007 IEEE Symposium on Security and Privacy (SP '07), 2007, pp. 321–334, http://dx.doi.org/10.1109/SP.2007.11.

[6] V. Goyal, O. Pandey, A. Sahai, B. Waters, Attribute-based encryption for fine-grained access control of encrypted data, in: Proceedings of the ACM Conference on Computer and Communications Security, Vol. 89–98, 2006, pp. 89–98, http://dx.doi.org/10.1145/1180405.1180418.

[7] Y. Xue, K. Xue, N. Gai, J. Hong, D.S.L. Wei, P. Hong, An attribute-based controlled collaborative access control scheme for public cloud storage, IEEE Trans. Inf. Forensics Secur. 14 (11) (2019) 2927–2942, http://dx.doi.org/10.1109/TIFS.2019.2911166.

[8] J. Hao, C. Huang, J. Ni, H. Rong, M. Xian, X.S. Shen, Fine-grained data access control with attribute-hiding policy for cloud-based iot, Comput. Netw. 153 (2019) 1–10, http://dx.doi.org/10.1016/j.comnet.2019.02.008, URL http://www.sciencedirect.com/science/article/pii/S1389128619301793.

[9] J. Li, N. Chen, Y. Zhang, Extended file hierarchy access control scheme with attribute based encryption in cloud computing, IEEE Trans. Emerg. Top. Comput. (2019) 1, http://dx.doi.org/10.1109/TETC.2019.2904637.

[10] M. Chase, Multi-authority attribute based encryption, in: Theory of Cryptography. LNCS, Vol. 4392, 2007, pp. 515–534, http://dx.doi.org/10.1007/978-3-540-70936-7_28.

[11] A. Lewko, B. Waters, Decentralizing attribute-based encryption, in: Advances in Cryptology – EUROCRYPT 2011. LNCS, Vol. 6632, 2011, pp. 568–588, http://dx.doi.org/10.1007/978-3-540-70936-7_28.

[12] K. Yang, X. Jia, K. Ren, B. Zhang, Dac-macs: Effective data access control for multi-authority cloud storage systems, in: 2013 Proceedings IEEE INFOCOM, 2013, pp. 2895–2903, http://dx.doi.org/10.1109/INFCOM.2013.6567100.

[13] V.K.A. Sandor, Y. Lin, X. Li, F. Lin, S. Zhang, Efficient decentralized multi-authority attribute based encryption for mobile cloud data storage, J. Netw. Comput. Appl. 129 (2019) 25–36, http://dx.doi.org/10.1016/j.jnca.2019.01.003.

[14] W. Li, K. Xue, Y. Xue, J. Hong, Tmacs: A robust and verifiable threshold multi-authority access control system in public cloud storage, IEEE Trans. Parallel Distrib. Syst. 27 (5) (2016) 1484–1496, http://dx.doi.org/10.1109/TPDS.2015.2448095.

[15] T. Salman, M. Zolanvari, A. Erbad, R. Jain, M. Samaka, Security services using blockchains: A state of the art survey, IEEE Commun. Surv. Tutor. 21 (1) (2019) 858–880, http://dx.doi.org/10.1109/COMST.2018.2863956.

[16] C. Ge, X. Ma, Z. Liu, A semi-autonomous distributed blockchain-based framework for uavs system, J. Syst. Archit. 107 (2020) 101728, http://dx.doi.org/10.1016/j.sysarc.2020.101728, URL http://www.sciencedirect.com/science/article/pii/S1383762120300229.

[17] J. Yang, Z. Lu, J. Wu, Smart-toy-edge-computing-oriented data exchange based on blockchain, J. Syst. Archit. 87 (2018) 36–48, http://dx.doi.org/10.1016/j.sysarc.2018.05.001.

[18] Q. Lyu, Y. Qi, X. Zhang, H. Liu, Q. Wang, N. Zheng, Sbac: A secure blockchain-based access control framework for information-centric networking, J. Netw. Comput. Appl. 149 (2020) 102444, http://dx.doi.org/10.1016/j.jnca.2019.102444.

[19] O.J.A. Pinno, A.R.A. Gregio, L.C.E. De Bona, Controlchain: Blockchain as a central enabler for access control authorizations in the iot, in: GLOBECOM 2017 - 2017 IEEE Global Communications Conference, 2017, pp. 1–6, http://dx.doi.org/10.1109/GLOCOM.2017.8254521.

[20] O. Samuel, N. Javaid, M. Awais, Z. Ahmed, M. Imran, M. Guizani, A blockchain model for fair data sharing in deregulated smart grids, in: 2019 IEEE Global Communications Conference (GLOBECOM), 2019, pp. 1–7, http://dx.doi.org/10.1109/GLOBECOM38437.2019.9013372.

[21] L. Zhu, Y. Wu, K. Gai, K.-K.R. Choo, Controllable and trustworthy blockchain-based cloud data management, Future Gener. Comput. Syst. 91 (2019) 527–535, http://dx.doi.org/10.1016/j.future.2018.09.019, URL http://www.sciencedirect.com/science/article/pii/S0167739X18311993.

[22] L. Chen, W.-K. Lee, C.-C. Chang, K.-K.R. Choo, N. Zhang, Blockchain based searchable encryption for electronic health record sharing, Future Gener. Comput. Syst. 95 (2019) 420–429, http://dx.doi.org/10.1016/j.future.2019.01.018, URL http://www.sciencedirect.com/science/article/pii/S0167739X18314134.

[23] P. Wei, D. Wang, Y. Zhao, S.K.S. Tyagi, N. Kumar, Blockchain data-based cloud data integrity protection mechanism, Future Gener. Comput. Syst. 102 (2020) 902–911, http://dx.doi.org/10.1016/j.future.2019.09.028.

[24] S. A., W. B., Fuzzy identity-based encryption, in: Advances in Cryptology - EUROCRYPT 2005.LNCS, Vol. 3494, 2005, pp. 457–473, http://dx.doi.org/10.1007/11426639_27.

[25] Y. Zhu, Y. Qin, Z. Zhou, X. Song, G. Liu, W.C. Chu, Digital asset management with distributed permission over blockchain and attribute-based access control, in: 2018 IEEE International Conference on Services Computing (SCC), 2018, pp. 193–200, http://dx.doi.org/10.1109/SCC.2018.00032.

[26] D.D.F. Maesa, P. Mori, L. Ricci, A blockchain based approach for the definition of auditable access control systems, Comput. Secur. 84 (2019) 93–119, http://dx.doi.org/10.1016/j.cose.2019.03.016.

[27] J. Paillisse, J. Subira, A. Lopez, A. Rodriguez-Natal, V. Ermagan, F. Maino, A. Cabellos, Distributed access control with blockchain, in: ICC 2019 - 2019 IEEE International Conference on Communications (ICC), 2019, pp. 1–6, http://dx.doi.org/10.1109/ICC.2019.8761995.

[28] H. Guo, E. Meamari, C.-C. Shen, Multi-authority attribute-based access control with smart contract, in: ICBCT 2019: Proceedings of the 2019 International Conference on Blockchain Technology, 2019, pp. 6–11, http://dx.doi.org/10.1145/3320154.3320164.

[29] S. Gao, G. Piao, J. Zhu, X. Ma, J. Ma, Trustaccess: A trustworthy secure ciphertext-policy and attribute hiding access control scheme based on blockchain, IEEE Trans. Veh. Technol. (2020) 1, http://dx.doi.org/10.1109/TVT.2020.2967099.

[30] S. Wan, M. Li, G. Liu, C. Wang, Recent advances in consensus protocols for blockchain: a survey, Wirel. Netw. (2019) http://dx.doi.org/10.1007/s11276-019-02195-0.

[31] H. Sukhwani, N. Wang, K.S. Trivedi, A. Rindos, Performance modeling of hyperledger fabric (permissioned blockchain network), in: 2018 IEEE 17th International Symposium on Network Computing and Applications (NCA), 2018, pp. 1–8, http://dx.doi.org/10.1109/NCA.2018.8548070.